

Project report for DGI

Water caustics using photon mapping

Lovisa Claesson

031019-2489

lovisacl@kth.se

Agnes Lindencrona

020211-8949

lindencr@kth.se

Abstract

This project studies the topic of water caustics and the effect of a Gaussian filter with different photon gathering radii. We worked mostly in Unity, where we rendered the caustics using photon mapping. The results show that when using photon gathering radii of 5, 6, and 7, the Gaussian filter does not improve the render. Furthermore, we found that these three radius sizes are the most suitable for our scenario. However, the Gaussian filter does improve the image when the photon gathering radius is outside this range.

1. Project specification

1.1. Background

An important aspect of how to display graphics in a computer is the representation of light. In the real world, we can only see the objects whose light rays reach the receptors in our eyes. When mimicking these behaviors digitally, there is usually a camera that represents the viewer, while light can be represented with a source and vectors.

In a digital space made up of cameras, scenes, objects, and vectors there are many methods of mimicking real light behavior. The rendering method we will discuss in this text is called *photon mapping* and this project is using photon mapping to see how the light creates caustics after refractions on water. The project further analyses how caustics is affected by the use of something called a *gaussian filter*.

We chose to do this project because we wanted to extend our knowledge on light refraction. Both of us find it fun to create nice visuals through different media. We figured that this project would help deepen our understanding of different light-qualities which would be of help for future rendering projects. Our initial plan was to study light in broken glass or colored glass, but once we started talking we find refraction in water to be more interesting.

1.2. Scenario

There is a great difference in how water caustics are expected to look depending on the environment of the scenario and the light source. It is therefore important to define a specific scenario for our project. Our chosen scenario is at the bottom of a square pool. The water surface represents the moment after something has been dropped into the middle of the pool, with sine waves extending from the point of impact. The light source is positioned close to the water surface, suggesting that a flashlight is being shone from the middle of the pool. We chose this scenario as it provides a simple way to create shapes in the water. Furthermore, it is one of the simpler scenarios to recreate in real life, which will facilitate the evaluation process.

1.3. Project focus

Our project is about implementing water caustics into a scene, with a focus on the use of gaussian filter and photons gathering to create a more smooth result.

1.4. Resources

Most of the implementation is done in Unity, version 6.4 and the scripts are done in C#. GitHub is used to keep track and share our progress on the project. lastly, a 3D rendering program called Blender was used to create the model of the water surface.

1.5. Similar implementations

A few projects that had done a similar study to ours were Karan bhagat project that he posted on [GitHub](#). He rendered light using photon mapping in Unity. Futher, Daniel Sánchez-Crespo and Juan Guardado rendered caustics using OpenGL and documented their process [1].

1.6. Plan overview

In order to create the project, our steps were

1. Gather information about photon mapping, water caustics, photon gathering and gaussian filter.
2. Create a model for a Cornell box with a single texture and the water in Blender.

3. Import the model and create the environment in Unity.
4. Create an algorithm for the refraction of the light and saving photons in the texture.
5. Create the photon gathering and gaussian filter.
6. Test the product and evaluate the results.

2. Theory

2.1. Photon mapping

Photon mapping is a rendering technique that is based on ray tracing. Classic ray tracing simulates viewing rays that originate from the camera. These rays are followed for one or more bounces on the environment objects. At each bounce it is determined if the coordinate is in light or shadow by checking if the light source can be reached without bounces.

Photon mapping is instead done in two passes. The first pass scatters photons throughout the environment and saves information on how and where they interact with the objects. The second pass originates as viewing vectors from the camera and determine which parts of the scene are visible and need to be considered. This pass uses the photon information from the first pass to determine the light intensity at each viewable point [2].

The typical data structure for photon mapping is a kd-tree, which is a data structure that allows data to be stored in a k-dimensional space. This method of saving data allows fast access to scene information. We opted for a method that would fit our setup more and simplify the implementation, which was storing the photons in the environment's texture.

2.2. Ray tracing

Ray tracing is a type of rendering technique that simulates the lighting in a scene. By shooting multiple rays from a light source the brightness and shadows can be calculated on the collided surface. The position of the camera is essential for calculating the resulting light [3].

2.3. Water caustics

If you are standing on a bridge above clear water and drop a rock into a pond below, you might notice a bright dancing pattern resulting from the waves of the rock. This type of pattern is what is referred to as water caustics. It is the unique reflection created when light refracts upon the water surface. Without water caustics the graphics feels a lot more flat and unbelievable. A clear example can be seen in [Figure 1](#) and [Figure 2](#). These show the before and after of caustics used to render a scene in the movie Moana.

You might notice that the second [Figure 2](#), feels a lot more believable, even if the change isn't much. In computer graphics this is simulated using different rendering techniques, such as ray tracing or photon mapping. There is not a "correct" way to represent water caustic, but different

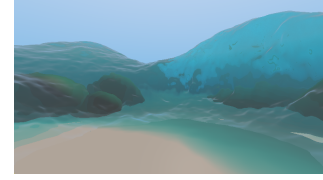


Figure 1. Before water caustics

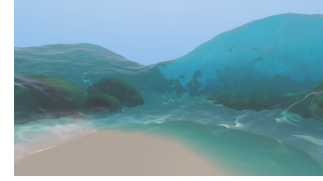


Figure 2. After water caustics

sources are all using different methods. However there are some similarities among all representations. One of this is the use of light rays and *Snell's law* to calculate the light refractions.

2.4. Snell's Law

Photons from a light source are refracted when they collide with a transparent surface. The direction the refracted photons are affected by the incoming angle and a characteristic of the material called the *refractive index*. This variable describes how much the light slows down once the photons enter the material. The index for air is 1.00029 and water 1.33333. In order to calculate the change in direction for the outgoing photon we use something called Snell's law. This is a mathematical law used to describe the relation between the incoming angle and the outgoing angle. The law is presented as follows

$$\eta_1 \cdot \sin\theta_1 = \eta_2 \cdot \sin\theta_2 \quad (1)$$

where θ_1 is the incoming angle, θ_2 is the outgoing angle, and η_1 and η_2 are the refractive indices of different materials the light passes through[4]. This is visualized in [Figure 3](#).

But this is non-trivial to write in code as it only imposes one restriction[1], therefore an approximation introduced by Foley et al. in 1996 [5] was used. This approximation is written as follows

$$T = N \cdot \left(\frac{\eta_1}{\eta_2} (E \cdot N) \pm \sqrt{1 + \left(\frac{\eta_1}{\eta_2} \right)^2 ((E \cdot N)^2 - 1)} \right) + \frac{\eta_1}{\eta_2} E \quad (2)$$

where E is the incoming ray, T is the outgoing ray, N is the surface normal, η_1 is the refractive index of the incoming material, and η_2 is the refractive index of the outgoing material.

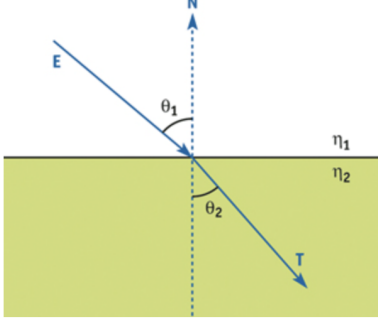


Figure 3. The relation between the angles of incoming and outgoing light vectors dependent on the refractive indices.

3. Photon Gathering

Simply displaying light where photons have hit a surface is not the entirety of the photon mapping algorithm. Since photons are small in reality our eyes cannot distinguish them individually. Therefore, in order to get a more cohesive light dispersion we use the density of photons in an area to estimate the reflected light intensity there. In order to compute the radiance at a point we start with the expression for reflected radiance:

$$L_r(x, \vec{w}) = \int_{\Omega_x} f_r(x, \vec{w}', \vec{w}) L_i(x, \vec{w}') |\vec{n}_x \cdot \vec{w}'| dw'_i \quad (3)$$

where L_r is the reflected radiance at point x in direction $\vec{w}$, Ω_x is the hemisphere around the point, f_r is the BRDF (bidirectional reflectance distribution function), and L_i is the incoming radiance. In Figure 4 this is visualized. The incoming radiance can be expressed in terms of the incoming flux $\Phi_i(x, \vec{w}')$

$$L_i(x, \vec{w}') = \frac{d^2\Phi_i(x, \vec{w}')}{\cos(\theta_i) dw'_i dA_i} \quad (4)$$

Putting equations Equation 3 and Equation 4 together we get the expression for reflected radiance in terms of flux.

$$L_r(x, \vec{w}) = \int_{\Omega_x} f_r(x, \vec{w}', \vec{w}) \frac{d^2\Phi_i(x, \vec{w}')}{dA_i} \quad (5)$$

This equation can then be simplified to an approximation as the sum over the n photons within the sphere around the point [6].

$$L_r(x, \vec{w}) = \sum_{p=1}^n f_r(x, \vec{w}_p, \vec{w}) \frac{\Delta\Phi_p(x, \vec{w}_p)}{\Delta A} \quad (6)$$

Finally, the area can be estimated as a circle around the point, which can be expressed in terms of a radius. Thus the reflected radiance equation is simplified to

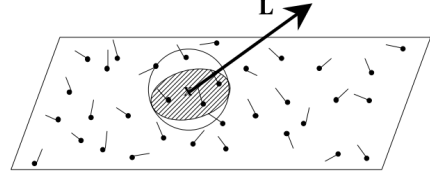


Figure 4. Radiance estimated from the nearby photons on the surface [7]

$$L_r(x, \vec{w}) \approx \frac{1}{\pi r^2} \sum_{p=1}^n f_r(x, \vec{w}_p, \vec{w}) \Delta\Phi_p(x, \vec{w}_p) \quad (7)$$

In other words, photon gathering is used to estimate how much light is present at a certain point on a surface by observing nearby photons. Every photon contributes to the estimation of the point's radiance, and the BRDF of the material determines how much of that energy reflects to the camera. Instead of integrating the incoming radiance over all possible directions we use the sum of the photons' light energy inside a certain radius. The bigger radius the more photons are used to estimate the light on that point. This usually reduces the noise, but could also make finer details less distinguishable [7].

3.1. Gaussian filter

The previously estimated reflected radiance Equation 7 has a drawback, if the number of photons is relatively low then the photon gathering can result in a blurry final radiance estimation. There are filtering methods that weight the contribution of each photon in the radiance estimation. One such filter is called a Gaussian filter [7], and it introduces a weighted term w_{pg} for each photon in the sum. The weight for each photon is defined as

$$w_{pg} = \alpha \cdot \left(1 - \frac{1 - e^{-\beta \frac{d_p^2}{2r^2}}}{1 - e^{-\beta}} \right) \quad (8)$$

Where d_p is the distance between the central photon and the neighboring photon, and the constants $\alpha = 0.918$ and $\beta = 1.953$ as defined by [8]. By adding this term to the radiance equation in Equation 7 the final algorithm becomes

$$L_r(x, \vec{w}) \approx \frac{1}{\pi r^2} \sum_{p=1}^n f_r(x, \vec{w}_p, \vec{w}) \Delta\Phi_p(x, \vec{w}_p) \cdot w_{pg} \quad (9)$$

The weight of each photon determines its contribution to the final radiance estimation based on its distance from the point being evaluated.

4. Implementation

4.1. Implementation details and assets

For this project we have used Unity3D for the rendering. Both of us have some limited prior experience with Unity3D that is not related to rendering. There are some resources for photon mapping on Unity online as well as implementation resources.

For the visualization of the photon map we decided to do a combination of techniques, both rendering the final camera rays directly in the Unity viewer and painting the texture of the object itself. The texture painting was informed by the decision to store the photons in the texture for fast lookup instead of a tree of coordinates.

For texture painting, the easiest implementation would not include the colors of the Cornell box. With this simplification we could wrap the entire box in one black texture to paint the light values on.

The water is modeled using blender and imported into Unity. We chose to create our own model as we had a difficult time finding a fitting scene online for free. Many of our early models found online included noticeable triangles in the texture, which was made very obvious when light refracted on the surface. However, blender is something that we thought was fun and found this opportunity to try and learn the workspace a little.

4.2. Setup

The implementation was done in Unity with the use of C# code and compute shaders. The C# code dispatches the compute shaders with rays on the GPU which reduced the execution time compared to running the code in C# on the CPU. This is done since there are many rays and this leads to a lot of calculations.

The scene contains two objects, one water mesh that is treated as a completely penetrable surface, so only the refraction is accounted for and not the reflection. The other object in the scene is the Cornell box. This object was changed in Blender to only have a single texture for the entire object, this was decided upon in order for the visualization to produce pixels of the same size for the entire box. If the pixels were not the same size then for different surfaces then parts of the object would not be rendered with as much detail as other parts.

The code three RenderTexture data structures used to store and display information. One of the textures stores the photon from the first photon pass and the other two are used to display the final texture both on the Unity object and in the Unity view port.

4.3. Algorithm

A brief overview of the algorithm can be found in [Algorithm 1](#).

First, the Unity objects in the scene are saved as collections of triangles containing the vertices of the triangles in world space, the corresponding uv coordinates and the normal of the triangle.

Then the compute shader is dispatched to do the first pass of photon mapping: distributing the photons in the scene. The photon rays are checked if they intersect with the water first and then with the Cornell box, this is done using the Trumbore intersection algorithm. If the ray hits water then the Barycentric coordinates from the hit are converted into world coordinates and are set as the new ray origin. The new ray direction is determined using Snell's law and the normal of the hit triangle. If the ray never hits the Cornell box then the photon is ignored, otherwise this barycentric coordinate is instead translated into a UV coordinates and saved in the hitCount texture.

Once all photon rays are cast, computed, and saved the screen rays from the camera are cast. Each ray is cast forward from the corresponding screen pixel and taking into account varying FOVs. If the ray does not hit a triangle from the Cornell box then it is not saved. But if a ray does hit a triangle then the barycentric coordinate of the hit is used to calculate the texture coordinates and find how many hits are saved there. The number of hits in the coordinate determine the brightness of that pixels and its surrounding pixels for both the texture painting and the rendering. Lastly, the object texture is applied to the Cornell box and the screen texture is converted to a Texture2D format and displayed in the Unity GUI.

Algorithm 1 C# Code Overview

```
1: photonCount  $\leftarrow$  Empty texture
2: screenRender  $\leftarrow$  Empty texture
3: textureRender  $\leftarrow$  Empty texture
4: groundTriangles  $\leftarrow$  Cornell box triangles
5: waterTriangles  $\leftarrow$  water triangles
6: Dispatch photon rays
7: Dispatch screen rays and gather photons
8: Apply texture to Cornell box
9: Convert and apply texture to GUI
```

5. Results and evaluation

An evaluation technique called *comparative evaluation* was chosen for our project. The technique is based on a comparison between two or more systems on different predetermined criteria [9]. In this case, we compare our simulated result to a real life image. Our goal is to evaluate if a gaussian filter creates a render more similar to a real life image compared to not using a gaussian filter. Therefore, we decided that an evaluation technique based on comparison was a suitable choice.

Figure 5 shows a rendered result, while Figure 6 is a photograph of the caustics of real water in a similar scenario. The photograph depicts water inside a glass container with a light source positioned directly above the water. The photograph was taken shortly after an object touched the center of the water surface, creating the waves and caustic patterns visible in Figure 6. However, the photograph and our rendered result are not exact replicas of each other. The waves are similar, but perfectly sinusoidal waves are not easily created with real water, and the light source in the photograph is not positioned perfectly in the center of the container. The photograph is included to provide a real-life reference for comparison with our rendered result and is still useful for this purpose even if the conditions are not exact.

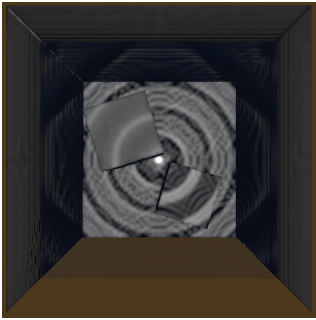


Figure 5. Screenshot of our render



Figure 6. Photo of water caustics

5.1. Overall evaluation

Our results show a caustic pattern that is similar to the one observed in the real-life image. The waves create alternating highlights and shadows in a circular pattern, which is formed by the waves in the water concentrating and dispersing the light. The render shows a highlight followed by a shadow, creating a circular pattern similar to that of the pho-

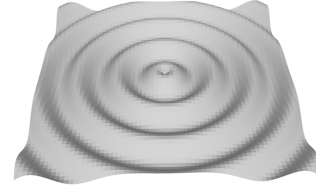


Figure 7. The mesh used for the water surface

tograph. However, our image contains a bright spot in the center that is not present in the real-life image. This bright spot is a result of the water model being convex around the center, as can be seen in Figure 7. Since the light source is positioned directly above this area, the incoming and outgoing directions of the photons are nearly aligned, causing the light to be concentrated in the center of the cornell box floor.

In the real-life example, the water surface and light source are more complex. The center of the caustic pattern does not have as large amplitude as the mesh, and the real surface likely contains small irregularities in its shape. Furthermore, the light source is not positioned exactly in the center of the container in the photograph. Since the photograph was taken while holding the light source by hand, its position may have introduced additional differences between the real-life example and our rendered result.

The waves in Figure 6 contain several important features that must remain clearly visible. These features are:

- The circular pattern
- The bright highlights
- The dark edges

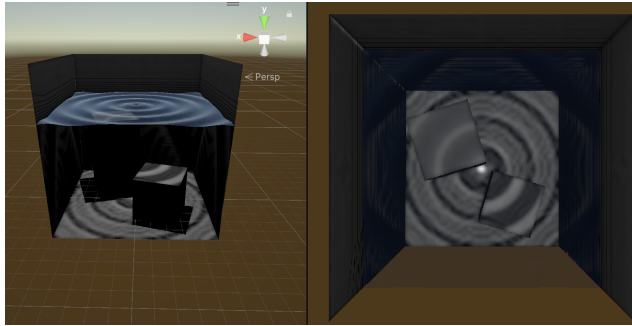
These features are based on our own observation. We decided these features were important as we were able to recreate the a similar image by only using these features.

5.2. Gaussian filter or not

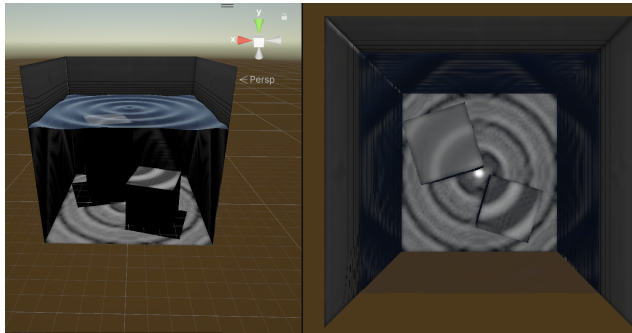
Since our study used two different functions, it also produced two different results. One result only uses photon gathering while the other includes a gaussian filter on top of that. By comparing them to the actual image and its key details, we can evaluate which function is best suited for photon caustics.

Additionally, it is important to determine which gathering radius is most appropriate, as different gathering radii gives different results. Depending on the size of the radius, the blend in both functions are affected. There is no universally correct radius size. A larger radius reduces more noise, but the downside is that details become less visible. It is therefore important to find a balance between noise reduction and detail preservation. We want the image to contain as little noise as possible while still preserving the key

features of the original image. The key features are the once described in the list of section 6.1.



(a) With gaussian filter

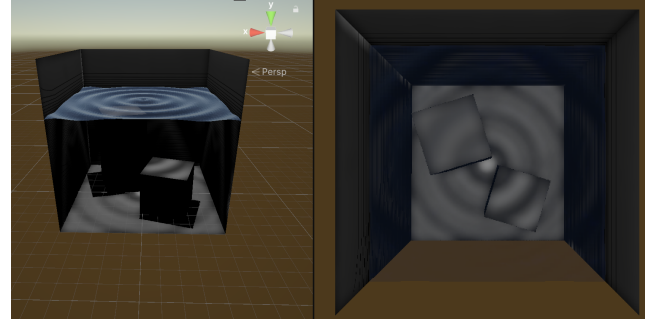


(b) Without gaussian filter

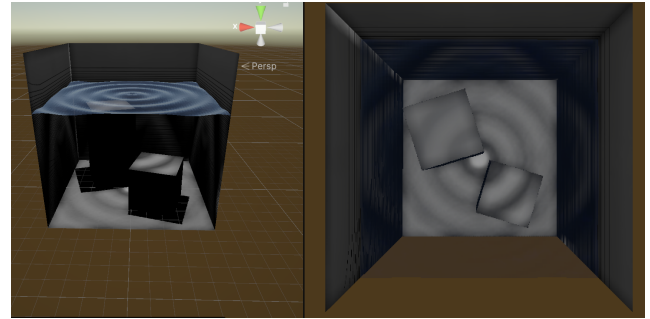
Figure 8. Radius 8

With a radius of 8, there is little noise in the image, but the details are more difficult to distinguish. As the radius increases further, the details become progressively less noticeable. This is true for both functions, but the one without gaussian filter it is slightly more difficult to distinguish the brightest highlights. This is due to the entire caustics being brighter, something that is obvious in Figure 8. Furthermore, the blend of the bright spots in the Figure 8b are bigger causing the darker circles to be smaller. This becomes more noticeable if we compare a very big radius, for example a radius of 20 in Figure 9. The difference is not major but the size of the darker circles are slightly smaller without the gaussian filter, causing them to be less noticeable. the difference in sizes of the circles can be seen in Figure 10, where gaussian filter is above and without gaussian filter below. Additionally the brightest highlights are barely visible in Figure 9b, but can still be seen in Figure 9a. Therefore if we were to use a big radius, then the function without gaussian filter is more suitable.

If we compare Figure 11b and Figure 11a using a smaller radius of 4 then the black circles are clearly visible in both functions. The noticeable difference is that the image without gaussian filter is much brighter than the image using gaussian filter. All of the important details are still visible in both figures, however due to Figure 11b being brighter it



(a) Gaussian



(b) Only gathering

Figure 9. Radius 20

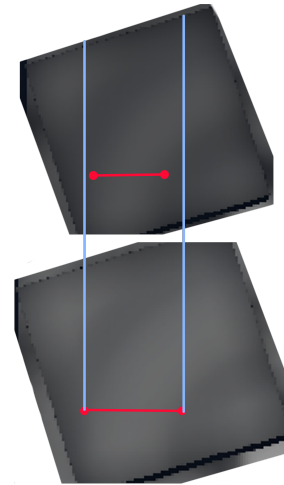
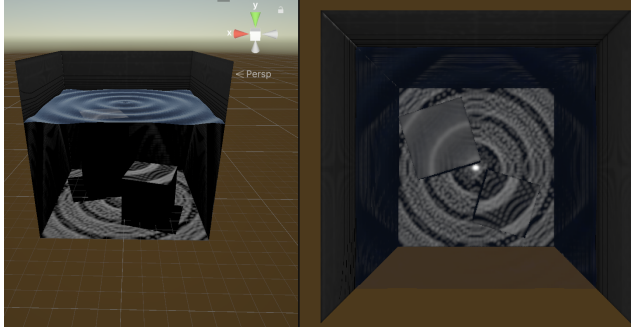


Figure 10. Enter Caption

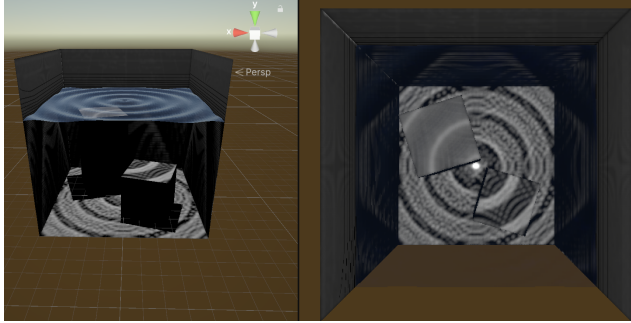
can be argued that the highlights are more difficult to spot at a first glance.

However, it is true for both results that a radius of 4 leaves a significant amount of noise in the image, although the details are easier to distinguish. With an even smaller radius the noise in the image is more visible.

A radius of 4 produces too much noise, but with a radius of 8 important details are starting to be less distinguishable. With radii between 5 and 7, the noise is reduced while the



(a) Gaussian



(b) Only gathering

Figure 11. Radius 4

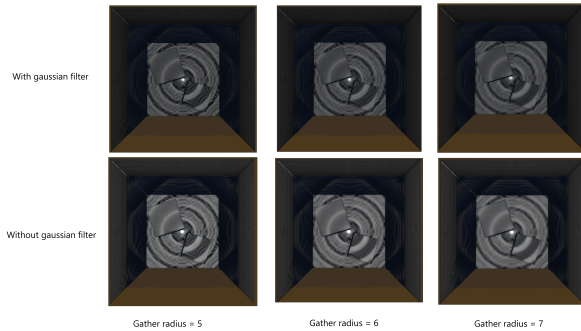


Figure 12. comparison between with gaussian filter and without

important details remain visible. Therefore, we argue that radii of 5, 6, and 7 provide a good balance between noise reduction and preservation of detail.

A comparison between the two functions using a radius of 5, 6 and 7 can be found in Figure 12. With these radii both the functions does not have much noise and the details are still clearly visible.

In conclusion, a gathering radius between 5 and 7 pixels is most appropriate when creating water caustics for a texture of size 1024×1024 pixels. Further, using a gather radius outside this spectrum, it is more appropriate to use

a gaussian filter, due to the details being less visible otherwise. However, inside the spectrum it is not as important which function to use as the key features are clearly visible using both functions.

6. Reflection

We encountered two main problems during the course of the project, with the first one being finding a suitable mesh for the water surface. Most of the open-source meshes had noticeable triangles in their textures. We therefore decided to recreate the mesh in Blender, which we saw as a learning opportunity. Something similar happened with the cornell box, the mesh had to be remade into a single object so we could cover it in one texture. Blender was also used for that task.

The second problem was the summer break. During the break, our communication was lacking, as our schedules rarely aligned. We therefore worked separately a lot without proper communication. Our solution was to have a meeting once a week to catch up on what we had been doing and work on issues together. We could have avoided this issue by having a plan for the project when the summer began, which would have made communication a lot easier.

6.1. Perceptual user studies

A perceptual user study is a study investigating how humans visually perceive and assess a result, for example how a rendered image looks like. Agnes have been part of a user study before where she got to stand in a virtual environment of a dark street. She got to express what environment felt most safe depending on the placement of various objects, such as doorways, cars, lampposts. This is a good example of how a Perceptual user study can be used to help developers creating something that makes the consumers feel secure and safe using their product. Instead of only evaluating the technical aspects of a product, a perceptual user study can provide information about how the result is actually experienced by the intended users.

This type of study would be helpful for our project as well. Both the functions using photon gathering and gaussian filter works well when using a radius between 5 and 7, but a user study could help to choose which of them is more suitable. This could be done in multiple ways. One way could be by having multiple participants looking at results from both methods and expressing which they preferred. If the majority of the participants exclaim they preferred one image above the other, then that method is what is best suited to use. Additionally our study could benefit by having a perceptual user study on what gather radius to use. The result of our study was that a gather radius of 5, 6 or 7 would be best suited for water caustics in our scene, but it might not be what is preferred by others. A “correct” way on what “should” look best is not always what is per-

ceived as visually preferred. In order to get data on what is perceived as most realistic/most preferred visual multiple participants could choose between a number of images of different radius. If a majority of the participants chooses one specific image, then that would be the preferred radius size.

However, there are some limitations to this type of evaluation. People's visual preferences are subjective and can vary depending on their previous experience, expectations, and individual perception. The results may also be influenced by factors such as the quality of the display or the order in which the images are presented. Therefore, a perceptual user study should not necessarily replace the technical evaluation, but rather complement it. Combining objective measurements with human perception would provide a more comprehensive evaluation of our rendering methods.

6.2. Contributions

We both worked on the initial Unity code together. Both worked on debugging the code together and the photon gathering was also done together. Both wrote parts of the report. Both have gathered sources to inform the theory the project is based on.

Lovisa started the work of migrating the initial code to a compute shader in order to improve speed by using the GPU instead. Lovisa mainly did the implementation details while Agnes wrote about and performed the evaluation. Additionally, Agnes wrote about the relevance of perceptual user studies on our project.

Generative AI was used to find some of the initial sources but most of them were not used as a reference in the final work. Some sources that were found using generative AI are:

- [10] for implementation details.
- [11] for the reference image that we based our scene on.

Another use of AI was to compile our sources into a biblatex file used to format the report.

References

- [1] J. Guardado and D. Sánchez-Crespo, "Rendering water caustics," in *GPU Gems*, NVIDIA Corporation, 2004, ch. 2. [Online]. Available: <https://developer.nvidia.com/gpugems/gpugems/part-i-natural-effects/chapter-2-rendering-water-caustics>.
- [2] Z. Waters, *Photon mapping*. Accessed: Aug. 23, 2026. [Online]. Available: https://web.cs.wpi.edu/~emmanuel/courses/cs563/write_ups/zackw/photon_mapping/PhotonMapping.html.
- [3] NVIDIA. "Ray tracing," Accessed: Aug. 29, 2026. [Online]. Available: <https://developer.nvidia.com/discover/ray-tracing>.
- [4] E. Mohn, "Snell's law," *EBSCO*, 2022. [Online]. Available: <https://www.ebsco.com/research-starters/law/snells-law>.
- [5] J. D. Foley, A. van Dam, S. K. Feiner, and J. F. Hughes, *Computer Graphics: Principles and Practice*, 2nd ed. Addison-Wesley, 1996.
- [6] M. Pharr, W. Jakob, and G. Humphreys, "The light transport equation," in *Physically Based Rendering: From Theory to Implementation*, 3rd ed. Morgan Kaufmann, 2018, ch. 14.4. Accessed: Aug. 23, 2026. [Online]. Available: https://pbr-book.org/3ed-2018/Light_Transport_I_Surface_Reflection/The_Light_Transport_Equation.
- [7] N. J. Christensen and H. W. Jensen, "A practical guide to global illumination using photon maps," in *SIGGRAPH 2000 Course 8*, Jul. 2000. Accessed: Aug. 23, 2026. [Online]. Available: <https://graphics.stanford.edu/courses/cs348b-00/course8.pdf>.
- [8] M. J. Pavićić, "Convenient anti-aliasing filters that minimize "bumpy" sampling," in *Graphics Gems*, A. S. Glassner, Ed., San Diego: Morgan Kaufmann, 1990, pp. 144–146, ISBN: 978-0-08-050753-8. DOI: <https://doi.org/10.1016/B978-0-08-050753-8.50041-3>.
- [9] P. Vartiainen, "On the principles of comparative evaluation," *Evaluation*, vol. 8, no. 3, pp. 359–371, 2002. DOI: [10.1177/135638902401462484](https://doi.org/10.1177/135638902401462484).
- [10] J. Flick. "Compute shaders," Accessed: Aug. 23, 2026. [Online]. Available: <https://catlikecoding.com/unity/tutorials/basics/compute-shaders/>.
- [11] M. Shah, J. Kontinen, and S. Pattanaik, "Caustics mapping: An image-space technique for real-time caustics," *IEEE Transactions on Visualization and Computer Graphics*, vol. 13, pp. 272–280, Mar. 2007. DOI: [10.1109/TVCG.2007.32](https://doi.org/10.1109/TVCG.2007.32).